

P2400R3

Library Evolution Report:

2021-09-28 to 2022-01-25

Published Proposal, 2022-04-21

Authors:

[Bryce Adelstein Lelbach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)
[Fabio Fracassi — Library Evolution Vice Chair](#) (CODE University of Applied Sciences)
[Ben Craig — Library Evolution Vice Chair](#) (NI)
[Billy Baker — Library Incubator Chair](#) (FlightSafety)
[Nevin Liber — Library Incubator Vice Chair](#) (Argonne National Laboratory)
[Inbal Levi — Ranges Chair \(SG9\) and Library Mailing List Review Manager](#)
[Corentin Jabot — Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1	Execution
1.1	C++23 Schedule
1.2	C++26 Schedule
1.3	Telecons
1.4	Mailing List Reviews
1.5	Paper Queues and Backlog
1.6	Electronic Polling
1.6.1	Expedited Advancement to Electronic Polling
1.7	Prioritization
1.8	Chair Guide

1.9 Staff

2 Technical

2.1 Plenary Approved Priorities

2.1.1 Senders And Receivers

2.1.2 Networking

2.1.3 Coroutines Library Support

2.1.4 Standard Library Modules

2.2 Other Highlights

2.2.1 Ranges

2.2.2 Formatting and Printing

2.2.3 Text and Unicode

2.2.4 Linear Algebra and `mdspan`

2.2.5 Concurrency TS v2

2.3 Papers Reviewed

2.4 Papers Advanced For C++23

References

Informative References

This report summarizes the activities of C++'s Library Evolution group from 2021-09-28 to 2022-01-25. It is split into two sections; one focused on *execution* (process, logistics, and operations) and one focused on *technical* work (papers we processed, etc).

Readers are encouraged to also look at:

- [\[P2145R1\]](#): Evolving C++ Remotely
- [\[P2138R3\]](#): Rules of Design $\Leftrightarrow$ Specification Engagement
- [\[P2195R2\]](#): Electronic Straw Polls
- [\[P1000R4\]](#): C++ Release Schedule
- [\[P0592R4\]](#): Plan and Priorities for C++23 and Beyond
- [\[P2489R0\]](#): Library Evolution Plan for Completing C++23

Prior Library Evolution reports:

- [\[P2247R1\]](#): 2020-04-06 to 2020-10-22
- [\[P2400R0\]](#): 2020-10-27 to 2021-02-16
- [\[P2400R1\]](#): 2021-02-23 to 2021-05-25
- [\[P2400R2\]](#): 2021-06-01 to 2021-09-20

§ 1. Execution

§ 1.1. C++23 Schedule

C++23 is now feature complete, as of 2022-02, as per our release schedule [\[P1000R4\]](#). No new features will be added to C++23. C++23 is the first C++ Standard designed entirely under our remote procedures [\[P2145R1\]](#).

We will continue to refine C++23 over the next few months. In particular, we'll focus on:

- Clarifications or questions from Library on material we've already forwarded to them for C++23.
- Papers that fix issues in papers we've sent to Library for C++23.
- Papers that fix issues in the Working Draft (especially issues we've recently introduced).

Library Evolution will continue working solely on C++23 through 2022-02, at which point we expect to complete all outstanding C++23 work.

In 2022-07, the C++ Committee will complete the C++23 Committee Draft (CD) and solicit comments from National Bodies (NBs). Once we receive those comments in 2022-Fall, Library Evolution will shift its attention back to C++23.

§ 1.2. C++26 Schedule

Library Evolution will begin working on C++26 in 2022-03, and C++26 will increasingly consume our attention as we approach the release of the C++23 Committee Draft.

By 2022-Summer, we expect to be fully focused on C++26. Our efforts will briefly shift away from C++26 in 2022-Fall so that we can address National Body comments and finalize C++23.

§ 1.3. Telecons

Since 2020-04, Library Evolution has held a 1.5 hour telecon once a week to review papers. Typically, 1 or 2 papers are reviewed at each telecon.

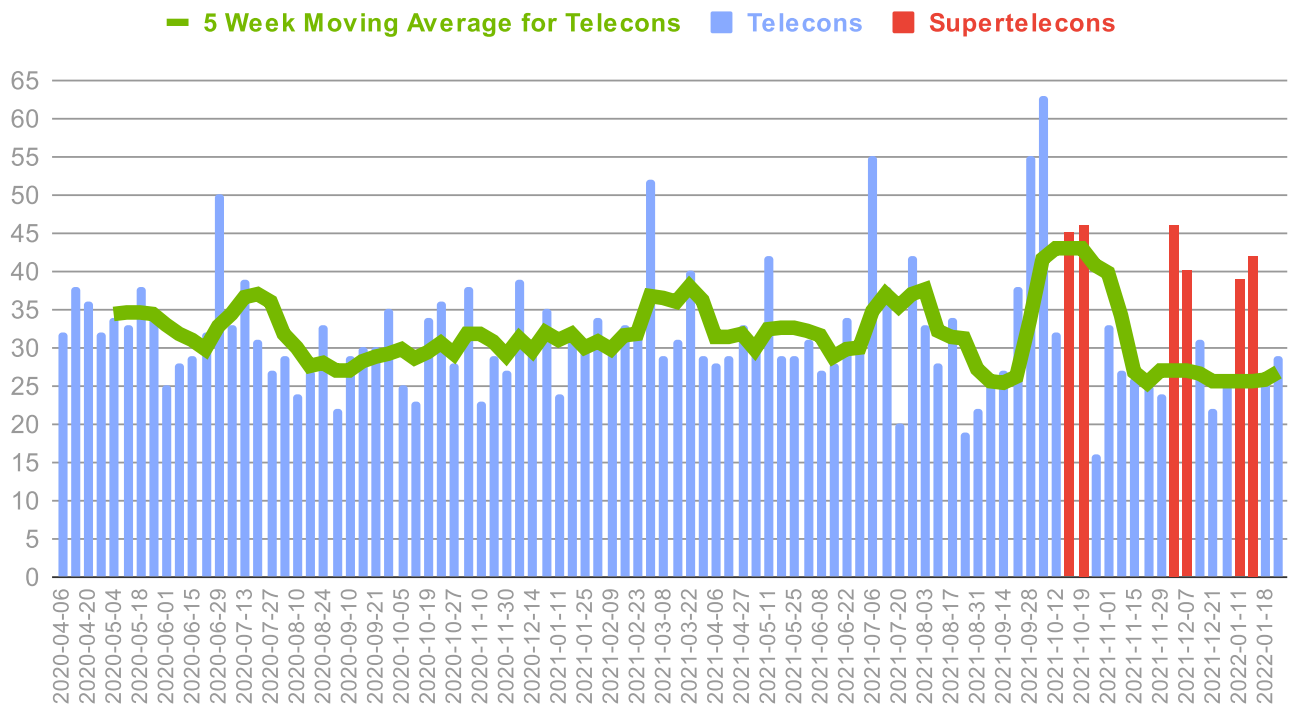
Recently, Library Evolution has started experimenting with a new format, supertelecons, to work on larger projects and help complete C++23. Supertelecons are held over 2 consecutive days, meeting for 3.5 hours a day, for 7 hours in total. We held three supertelecons - 2021-10, 2021-11, and 2022-01. These initial 3 supertelecons were quite successful, and we anticipate using this format alongside regular weekly telecons and mailing list reviews in the future.

The agenda for upcoming telecon reviews can be found [here](#). For details on how to participate in telecon reviews, please view the calendar on documents.isocpp.org or [contact Bryce Adelstein Lelbach](#).

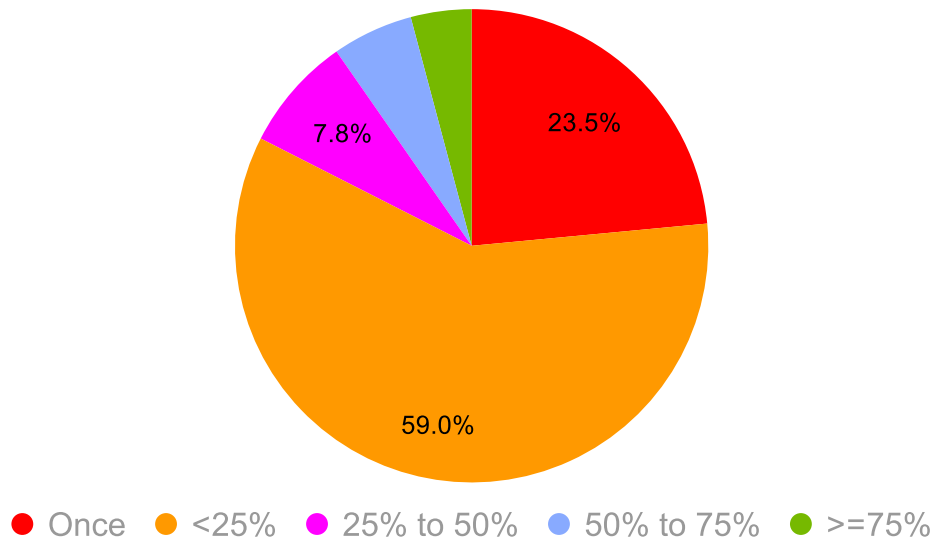
More details on how we use telecons can be found in [\[P2145R1\]](#).

	2021-09-28 to 2022-01-25	Since 2020-04-06
# of Telecons	14	88
# of Supertelecons	3	3
# of (Super)Telecon Days	6	6
# of Papers Reviewed On (Super)Telecons	36	143
# of Papers Advanced From (Super)Telecons to Electronic Polling	21	49
Total # of (Super)Telecon Attendees	121	217
Mean Attendees Per Telecon	34.65	31.58
Median Attendees Per Telecon	31.5	30.0
Mean Telecons Per Attendee	5.73	13.05
Median Telecons Per Attendee	3	4
Mean Attendees Per Supertelecon Day	43	43
Median Attendees Per Supertelecon Day	43.5	43.5

Library Evolution Telecon Attendance



Library Evolution Telecon Participation Per Attendee



§ 1.4. Mailing List Reviews

In addition to telecons, we review some papers via mailing list discussions.

Mailing list reviews are critical to our operations, as they allow us to increase our paper-processing bandwidth without increasing the frequency or duration of telecons. Additionally, certain classes of papers tend to be

particularly well suited for mailing list reviews, such as new proposals which tend to need feedback, not decision making.

We conduct 2 to 3 mailing list reviews concurrently, and each review lasts for a few weeks.

The agenda for upcoming mailing list reviews can be found [here](#). For details on how to participate in mailing list reviews, please [contact Bryce Adelstein Lelbach](#).

More details on how we use mailing lists can be found in [P2145R1](#).

	2021-09-28 to 2022-01-25	Since 2020-04-06
# of Papers Reviewed On The Mailing List	17	52
# of Papers Advanced From Mailing List Directly To Electronic Polling	18	21

§ 1.5. Paper Queues and Backlog

Queue	# of Papers
Active	44
C++23	15
C++26	34
Ready for Triage	0
Deferred	7
Ready for Mailing List Review	14
Scheduled for Mailing List Review	6
Ready for Meeting Review	8
Scheduled for Meeting Review	5
Expedited Electronic Poll Candidates	0
Ready for Electronic Poll	13
Scheduled for Electronic Poll	0

The above information is as of the publication of this paper. For up to date counts and more details on the various queues of papers being handled by Library Evolution, please see [the paper queries GitHub wiki page](#).

§ 1.6. Electronic Polling

Library Evolution typically conducts a quarterly series of electronic polls to affirm the decisions we make. From 2021-09 through 2022-01, we instead conducted electronic polls every month to accomodate our increased activity at the end of the C++23 design cycle.

The following polling periods have been conducted over the past year:

- 2020-Fall - completed, outcomes can be found in [P2262R0](#).
- 2021-Winter - completed, outcomes can be found in [P2368R0](#).
- 2021-Spring - completed, outcomes can be found in [P2384R1](#).
- 2021-Summer - completed, outcomes can be found in [P2435R1](#).
- 2021-09 - completed, outcomes can be found in [P2451R0](#).
- 2021-10 - completed, outcomes can be found in [P2453R0](#).
- 2021-11 - completed, outcomes can be found in [P2455R0](#).
- 2021-12 - completed, outcomes can be found in [P2457R0](#).
- 2022-01 - completed, outcomes can be found in [P2459R0](#).

More details on electronic polling can be found in [P2195R2](#).

§ 1.6.1. Expedited Advancement to Electronic Polling

Library Evolution telecons are the best way for us to review papers remotely, but our telecon time is limited. We started mailing list reviews of papers to reduce the demands on Library Evolution telecon time, but until recently, all papers have still required some telecon review before they are advanced to electronic polls and out of Library Evolution.

In the last few months, we have begun experimenting with advancing papers directly from mailing list review to electronic polling. The procedure is intended for smaller matters, non-controversial matters, and matters that we believe will not benefit from telecon discussion. It works as follows:

0.) A mailing list thread is started on lib-ext@ proposing that a paper be advanced directly to electronic polling. Background on the paper, its maturity, and the outcomes of previous discussions is provided. 1.) Participants are asked to take a stance on the motion by replying with either a "+1" or a "-1". 2.) When the thread reaches its nature conclusion, the chairs determine whether it is suitable for the paper to proceed to electronic polling. Typically, we are looking for strong and vocal support; a lack of objections is not sufficient.

§ 1.7. Prioritization

In conjunction with Library, we've prioritized all the papers that have been forwarded to Library, using the priority levels identified in the [plan and priorities for C++23 and beyond](#). We plan to update this prioritization in the coming weeks to help Library plan their efforts for completing C++23. See the [prioritized list of papers](#) for details.

§ 1.8. Chair Guide

We have developed a Library Evolution chairing guide, which can be found on the [Library Evolution GitHub wiki](#)

§ 1.9. Staff

Library Evolution is organized by a team, not a single individual:

- Bryce Adelstein Lelbach, Chair
- Fabio Fracassi, Vice Chair
- Ben Craig, Vice Chair
- Billy Baker, Incubator Chair
- Nevin Liber, Incubator Vice Chair
- Corentin Jabot, Mailing List Review Manager
- Inbal Levi, Mailing List Review Manager and Algorithms Task Force Chair
- Gašper Ažman, Coroutines Library Task Force Chair

Leadership by a group instead of a team has a number of benefits:

- Increases our capacity and capabilities.
- Reduces the burden on each leader.
- Builds institutional knowledge and redundancy.

We hold a regular staff meeting every two weeks to discuss our operations and make plans for the future.

Additionally, the following noble individuals take minutes for Library Evolution, and deserve accolades:

- Ben Craig
- Inbal Levi
- Mark Hoemmen
- Guy Davidson
- Dietmar Kühl
- Gašper Ažman
- Steve Downey
- Bronek Kozicki

§ 2. Technical

§ 2.1. Plenary Approved Priorities

The C++ Committee adopted a [plan and priorities for C++23 and beyond](#). This section describes the status of the priority Standard Library features described in that plan.

Plenary Approved Priority	Status
Executors	Under Library Evolution review, on track for C++26.
Networking	Under Networking Study Group review.
Coroutines Library Support	[P2502R1] <code>std::generator</code> design approved for C++23.
Standard Library Modules	[P2465R1] Standard Library Modules <code>std</code> And <code>std.compat</code> design approved for C++23.

§ 2.1.1. Senders And Receivers

In 2021-09 and 2021-10, Library Evolution discussed and compared two different models for asynchrony in the Standard Library: the Senders and Receivers model [\[P2300R4\]](#) and the Networking TS model [\[P2444R0\]](#). That discussion reached a few key conclusions [\[P2453R0\]](#):

- Library Evolution wants a single unifying model for asynchrony for the C++ Standard Library.
- There is consensus in Library Evolution that Senders and Receivers model is a good basis for most asynchronous use cases, including networking, parallelism, and heterogeneous computing.
- Most in Library Evolution feel that the Networking TS model is not a suitable basis for most asynchronous use cases, including parallelism and heterogeneous computing.
- Library Evolution does not want to stop working on the Networking TS.
- Library Evolution would like networking in the Standard Library to be based on Senders and Receivers.

Following those conclusions, Library Evolution focused on trying to complete [\[P2300R4\]](#) Senders and Receivers for C++23. We held three supertelecons focused on Senders and Receivers.

We made great progress on [\[P2300R4\]](#) Senders and Receivers over the last few months, and we have consensus in favor of the design. There is sustained strong opposition against including such a large proposal into C++23 at such a late stage. It is also unclear whether we would be able to complete wording review in the limited time we have available. Therefore, Library Evolution decided to not advance [\[P2300R4\]](#) Senders and Receivers for C++23. Instead, we hope to send it to Library early in the C++26 cycle.

§ 2.1.2. Networking

We discussed the Networking Technical Specification, [\[N4771\]](#), in 2021-Fall, reaching a series of conclusions in the [\[P2453R0\]](#) 2022-10 Library Evolution Poll Outcomes.

The Networking TS we have today uses a different asynchrony model from [\[P2300R4\]](#) Senders and Receivers. Library Evolution has expressed a desire to have a single model for asynchrony in the Standard Library and has consensus in favor of the Senders and Receivers design. If we are to proceed with Networking TS design as-is,

Library Evolution will need to be shown compelling new information on why networking should use a different asynchrony model.

Alternatively, we could pursue a refinement of the Networking TS based on senders or receivers or a different networking design altogether. Library Evolution strongly encourages work in this area, but we have no such proposal today.

Given the need for significant design changes, networking will not make it into C++23.

§ 2.1.3. Coroutines Library Support

A new proposal for `std::generator`, [\[P2502R1\]](#), was brought to Library Evolution in 2021-12. Many of the open design questions were addressed, however some key ones remained, such as what the reference type of `std::generator<T>` should be.

Library Evolution leadership formed an ad-hoc task force, led by Gašper Ažman, to discuss and try and resolve these remaining questions. This task force was successful, and in 2022-01, Library Evolution design approved [\[P2502R1\]](#) `std::generator` for C++23 via electronic poll.

§ 2.1.4. Standard Library Modules

In 2021-10, Library Evolution completed our work on [\[P2465R1\]](#) Standard Library Modules for C++23. After a lengthy discussion, we determined we liked the shape of the two modules in the proposal and we came up with names that we had consensus on. In 2021-11, [\[P2465R1\]](#) Standard Library Modules `std` and `std.compat` was design approved for C++23 via electronic poll.

§ 2.2. Other Highlights

§ 2.2.1. Ranges

We've delivered a significant amount of the work laid out in [\[P2214R2\]](#) A Plan for C++23 Ranges. Almost all of the tier 1 components are on track for C++23; for more details, see [\[P2214R2\]](#).

The following ranges proposals were advanced to Library Working Group for C++23 by electronic poll:

- [\[P2387R2\]](#): Pipe Support For User-Defined Range Adaptors
- [\[P2443R0\]](#): `views::chunk_by`
- [\[P2442R0\]](#): `views::chunk` And `views::slide`
- [\[P2374R3\]](#): `views::cartesian_product`
- [\[P2441R1\]](#): `views::join_with`
- [\[P2446R1\]](#): `views::all_move`

- [\[P1899R2\]](#): `views::stride`
- [\[P2474R1\]](#): `views::repeat`
- [\[P2440R0\]](#): `ranges::iota`, `ranges::shift_left`, And `ranges::shift_right`
- [\[P2322R5\]](#): `ranges::fold`
- [\[P2302R2\]](#): `ranges::contains`
- [\[P2278R2\]](#): `cbegin` Should Always Return A Constant Iterator
- [\[P2408R4\]](#): Ranges Iterators As Inputs To Non-Ranges Algorithms
- [\[P2494R1\]](#): Relaxing Range Adaptors To Allow For Move Only Types
- [\[P2165R3\]](#): Compatibility Between `tuple`, `pair`, And `tuple`-like objects
- [\[P2255R2\]](#): A Type Trait To Detect Reference Binding To Temporary

§ 2.2.2. Formatting and Printing

We've accomplished most of our goals for formatting and printing in C++23. We finished our review of the two major proposals, [\[P2093R13\]](#) Formatted Output and [\[P2286R6\]](#) Formatting Ranges, as well as [\[P2419R1\]](#) Clarify Handling Of Encodings In Localized Formatting Of Chrono Types, and advanced all three proposals to Library Working Group for C++23 via electronic poll.

§ 2.2.3. Text and Unicode

We completed our review of [\[P1885R8\]](#) Naming Text Encodings To Demystify Them, which is a part of the roadmap laid out in [\[P1238R0\]](#) Unicode Direction, and sent it to Library Working Group for C++23 via electronic poll. We also advanced [\[P2460R0\]](#) Relax Requirements On `wchar_t` To Match Existing Practices to Library Working Group for C++23 via electronic poll, although it still needs to be reviewed and approved by Language Evolution.

§ 2.2.4. Linear Algebra and `mdspan`

After many years of work, we finished reviewing [\[P0009R16\]](#) `mdspan` and sent it to Library Working Group for C++23. We plan to look at two follow-on papers that fix some issues in `mdspan` in the next few weeks: [\[P2554R0\]](#) C-Array Interoperability Of `mdspan` and [\[P2553R0\]](#) Make `mdspan` `size_type` Controllable. We'll also be reviewing [\[P1684R1\]](#) `mdarray`, an owning multidimensional analog of `mdspan` that is similar to a container adaptor.

We're planning to start another review of [\[P1673R6\]](#) Linear Algebra in 2022-Spring. The review of the other proposal in this space, [\[P1385R6\]](#), has been postponed at the request of its authors.

§ 2.2.5. Concurrency TS v2

We've been working on the Concurrency TS v2 for awhile, and it's finally nearing completion. A plan for completing it is outlined in [\[P2559R0\]](#).

The following proposals were advanced to Library Working Group for the Concurrency TS v2 by electronic poll:

- [\[P0493R3\]](#): Atomic Maximum/Minimum
- [\[P2396R0\]](#): Concurrency TS v2 Fixes

§ 2.3. Papers Reviewed

The following papers were evaluated in the 2021-10 Library Evolution polling period (see [\[P2453R0\]](#) for details):

- [\[P2300R4\]](#): `std::execution`
- [\[P2444R0\]](#): Asio Async Model

The following papers were advanced to to Library Working Group for C++23 in the 2021-11 Library Evolution polling period (see [\[P2455R0\]](#) for details):

- [\[P2465R1\]](#): Standard Library Modules `std` And `std.compat`
- [\[P2387R2\]](#): Pipe Support For User-Defined Range Adaptors
- [\[P2443R0\]](#): `views::chunk_by`
- [\[P2442R0\]](#): `views::chunk` And `views::slide`
- [\[P2440R0\]](#): `ranges::iota`, `ranges::shift_left`, And `ranges::shift_right`
- [\[P2255R2\]](#): A Type Trait To Detect Reference Binding To Temporary
- [\[P1885R8\]](#): Naming Text Encodings To Demystify Them
- [\[P2419R1\]](#): Clarify Handling Of Encodings In Localized Formatting Of Chrono Types
- [\[P2460R0\]](#): Relax Requirements On `wchar_t` To Match Existing Practices
- [\[P2445R0\]](#): `forward_like`
- [\[P2417R0\]](#): A More `constexpr` `bitset`
- [\[P1841R1\]](#): Wording For Individually Specializable Numeric Traits
- [\[P0627R6\]](#): `unreachable`

The following papers were advanced to to Library Working Group for C++23 in the 2021-12 Library Evolution polling period (see [\[P2457R0\]](#) for details):

- [\[P0009R16\]](#): `mdspan`
- [\[P2093R13\]](#): Formatted Output

- [\[P1467R7\]](#): Extended Floating-Point Types
- [\[P2278R2\]](#): `cbegin` Should Always Return A Constant Iterator
- [\[P2408R4\]](#): Ranges Iterators As Inputs To Non-Ranges Algorithms
- [\[P2374R3\]](#): `views::cartesian_product`
- [\[P2441R1\]](#): `views::join_with`
- [\[P2446R1\]](#): `views::all_move`
- [\[P2467R0\]](#): Support Exclusive Mode For `fstreams`
- [\[P2438R1\]](#): `string::substr()` &&

The following papers were advanced to to Library Working Group for the Concurrency TS v2 in the 2021-12 Library Evolution polling period (see [\[P2457R0\]](#) for details):

- [\[P2396R0\]](#): Concurrency TS v2 Fixes

The following papers were evaluated in the 2021-12 Library Evolution polling period (see [\[P2457R0\]](#) for details):

- [\[P2198R3\]](#): Freestanding Feature-Test Macros And Implementation-Defined Extensions

The following papers were advanced to to Library Working Group for C++23 in the 2022-01 Library Evolution polling period (see [\[P2459R0\]](#) for details):

- [\[P2363R3\]](#): Extending Associative Containers With The Remaining Heterogeneous Overloads
- [\[P0493R3\]](#): Atomic Maximum/Minimum
- [\[P2286R6\]](#): Formatting Ranges
- [\[P2165R3\]](#): Compatibility Between `tuple`, `pair`, And `tuple`-like objects
- [\[P2494R1\]](#): Relaxing Range Adaptors To Allow For Move Only Types
- [\[P2322R5\]](#): `ranges::fold`
- [\[P2302R2\]](#): `ranges::contains`
- [\[P1899R2\]](#): `views::stride`
- [\[P2474R1\]](#): `views::repeat`
- [\[P2508R1\]](#): Expose `basic-format-string<charT, Args...>`
- [\[P2248R4\]](#): Enabling List-Initialization For Algorithms
- [\[P2404R2\]](#): Move-Only Types For `equality_comparable_with`, `totally_ordered_with`, And `three_way_comparable_with`
- [\[P2502R1\]](#): `std::generator`

The following papers were evaluated in the 2022-01 Library Evolution polling period (see [\[P2459R0\]](#) for details):

- [\[P2300R4\]](#): `std::execution`

The following papers were reviewed at Library Evolution telecons between 2021-09-28 and 2022-01-25:

- [\[P2300R4\]](#): `std::execution`
- [\[P2444R0\]](#): Asio Async Model
- [\[P2464R0\]](#): Ruminations On Networking And Executors
- [\[P2469R0\]](#): Response To P2464: The Networking TS Is Baked, P2300 Senders And Receivers Is Not
- [\[P2465R1\]](#): Standard Mibrary Modules `std` And `std.compat`
- [\[P2442R0\]](#): `views::chunk` And `views::slide`
- [\[P2443R0\]](#): `views::chunk_by`
- [\[P2374R3\]](#): `views::cartesian_product`
- [\[P2446R1\]](#): `views::all_move`
- [\[P2441R1\]](#): `views::join_with`
- [\[P2474R1\]](#): `views::repeat`
- [\[P2440R0\]](#): `ranges::iota`, `ranges::shift_left`, And `ranges::shift_right`
- [\[P2322R5\]](#): `ranges::fold`
- [\[P2387R2\]](#): Pipe Support For User-Defined Range Adaptors
- [\[P2494R1\]](#): Relaxing Range Adaptors To Allow For Move Only Types
- [\[P2255R2\]](#): A Type Trait To Detect Reference Binding To Temporary
- [\[P2278R2\]](#): `cbegin` Should Always Return A Constant Iterator
- [\[P0009R14\]](#): `mdspan`
- [\[P1885R8\]](#): Naming Text Encodings To Demystify Them
- [\[P2445R0\]](#): `forward_like`
- [\[P1467R7\]](#): Extended Floating Point Types
- [\[P2286R6\]](#): Formatting Ranges
- [\[P2093R13\]](#): Formatted Output
- [\[P2198R3\]](#): Freestanding Feature-Test Macros And Implementation-Defined Extensions
- [\[P2502R1\]](#): `std::generator`
- [\[P2529R0\]](#): `std::generator` Should Have `T&& reference_type`
- [\[P0493R3\]](#): Atomic Maximum/Minimum
- [\[P2486R1\]](#): Structured Naming For Function Object And CPO Values
- [\[P2165R3\]](#): Compatibility Between `tuple`, `pair`, And `tuple`-Like Objects
- [\[P2508R1\]](#): Expose `basic-format-string<charT, Args...>`

The following papers were reviewed at Library Evolution 2021-10 supertelecon:

- [\[P2300R4\]](#): `std::execution`
- [\[P2322R5\]](#): `ranges::fold`
- [\[P2440R0\]](#): `ranges::iota`, `ranges::shift_left`, And `ranges::shift_right`
- [\[P2387R2\]](#): Pipe Support For User-Defined Range Adaptors

The following papers were reviewed at Library Evolution 2021-11 supertelecon:

- [\[P2300R4\]](#): `std::execution`

The following papers were reviewed at Library Evolution 2022-01 supertelecon:

- [\[P2300R4\]](#): `std::execution`

The following papers were reviewed on the Library Evolution mailing list between 2021-09-28 and 2022-01-25:

- [\[P2407R0\]](#): Freestanding Library: Partial Classes
- [\[P1899R2\]](#): `views::stride`
- [\[P2302R2\]](#): `ranges::contains`
- [\[P2447R0\]](#): `span` And The Missing Constructor
- [\[P2408R4\]](#): Ranges Iterators As Inputs To Non-Ranges Algorithms
- [\[P2248R4\]](#): Enabling List-Initialization For Algorithms
- [\[P2405R0\]](#): `nullopt_t` And `nullptr_t` Should Both Have `operator<=>` And `operator==`
- [\[P2249R1\]](#): Mixed Comparisons For Smart Pointers
- [\[P2404R2\]](#): Move-Only Types For `equality_comparable_with`, `totally_ordered_with`, And `three_way_comparable_with`
- [\[P2438R1\]](#): `string::substr()` &&
- [\[P2363R3\]](#): Extending Associative Containers With The Remaining Heterogeneous Overloads
- [\[P2419R1\]](#): Clarify Handling Of Encodings In Localized Formatting Of Chrono Types
- [\[P2460R0\]](#): Relax Requirements On `wchar_t` To Match Existing Practices
- [\[P2467R0\]](#): Support Exclusive Mode For `fstreams`
- [\[P1841R1\]](#): Wording For Individually Specializable Numeric Traits
- [\[P0627R6\]](#): `unreachable`
- [\[P2396R0\]](#): Concurrency TS v2 Fixes

§ 2.4. Papers Advanced For C++23

Library Evolution advanced 64 papers to Library for C++23 since 2020-04-06:

- [\[P2363R3\]](#): Extending Associative Containers With The Remaining Heterogeneous Overloads
- [\[P0493R3\]](#): Atomic Maximum/Minimum
- [\[P2286R6\]](#): Formatting Ranges
- [\[P2165R3\]](#): Compatibility Between `tuple`, `pair`, And `tuple`-Like Objects
- [\[P2494R1\]](#): Relaxing Range Adaptors To Allow For Move Only Types
- [\[P2322R5\]](#): `ranges::fold`
- [\[P2302R2\]](#): `ranges::contains`
- [\[P1899R2\]](#): `views::stride`
- [\[P2474R1\]](#): `views::repeat`
- [\[P2508R1\]](#): Expose `basic-format-string<charT, Args...>`
- [\[P2248R4\]](#): Enabling List-Initialization For Algorithms
- [\[P2404R2\]](#): Move-Only Types For `equality_comparable_with`, `totally_ordered_with`, And `three_way_comparable_with`
- [\[P2502R1\]](#): `std::generator`
- [\[P0009R14\]](#): `mdspan`
- [\[P2093R13\]](#): Formatted Output
- [\[P1467R7\]](#): Extended Floating-Point Types
- [\[P2278R2\]](#): `cbegin` Should Always Return A Constant Iterator
- [\[P2408R4\]](#): Ranges Iterators As Inputs To Non-Ranges Algorithms
- [\[P2374R3\]](#): `views::cartesian_product`
- [\[P2441R1\]](#): `views::join_with`
- [\[P2446R1\]](#): `views::all_move`
- [\[P2467R0\]](#): Support Exclusive Mode For `fstreams`
- [\[P2438R1\]](#): `string::substr()` &&
- [\[P2465R1\]](#): Standard Library Modules `std` And `std.compat`
- [\[P2387R2\]](#): Pipe Support For User-Defined Range Adaptors
- [\[P2443R0\]](#): `views::chunk_by`
- [\[P2442R0\]](#): `views::chunk` And `views::slide`
- [\[P2440R0\]](#): `ranges::iota`, `ranges::shift_left`, And `ranges::shift_right`
- [\[P2255R2\]](#): A Type Trait To Detect Reference Binding To Temporary
- [\[P1885R8\]](#): Naming Text Encodings To Demystify Them
- [\[P2419R1\]](#): Clarify Handling Of Encodings In Localized Formatting Of Chrono Types

- [\[P2460R0\]](#): Relax Requirements On `wchar_t` To Match Existing Practices
- [\[P2445R0\]](#): `forward_like`
- [\[P2417R0\]](#): A More `constexpr` bitset
- [\[P2485R0\]](#): Do Not Add `value_exists` And `value_or` To C++23
- [\[P0627R6\]](#): Function To Mark Unreachable Code
- [\[P2418R0\]](#): Add Support For `std::generator`-like Types To `std::format`
- [\[P2415R1\]](#): What Is A view?
- [\[P2432R0\]](#): Fix `istream_view`
- [\[P2351R0\]](#): Mark All Library Static Cast Wrappers As `[[nodiscard]]`
- [\[P2291R2\]](#): Add `constexpr` Modifiers To Functions `to_chars` And `from_chars` For Integral Types In `<charconv>` Header
- [\[P2372R1\]](#): Fixing Locale Handling In Chrono Formatters
- [\[P1206R6\]](#): `ranges::to`
- [\[P0533R9\]](#): `constexpr` For `<cmath>` And `<cstdlib>`
- [\[P2273R2\]](#): Making `unique_ptr` `constexpr`
- [\[P0323R10\]](#): `expected`
- [\[P2325R2\]](#): Views Should Not Be Required To Be Default Constructible
- [\[P2328R0\]](#): `join_view` Should Join All Views Of Ranges
- [\[P2210R2\]](#): Superior String Splitting
- [\[P2321R1\]](#): `views::zip`
- [\[P2251R1\]](#): Require `span` & `basic_string_view` To Be Trivially Copyable
- [\[P1072R7\]](#): `basic_string::resize_and_overwrite`
- [\[P2340R0\]](#): Clarifying The Status Of The "C Headers"
- [\[P2301R0\]](#): Add A `pmr` Alias For `stacktrace`
- [\[P2265R0\]](#): Renaming `any_invocable`
- [\[P1642R5\]](#): Freestanding [utilities], [ranges], And [iterators]
- [\[P2216R2\]](#): `std::format` Improvements
- [\[P2077R2\]](#): Heterogeneous Erasure Overloads For Associative Containers
- [\[P2136R2\]](#): `invoke_r`
- [\[P1951R0\]](#): Default Arguments For `pair`'s Forwarding Constructor
- [\[P2231R0\]](#): Missing `constexpr` In `optional` And `variant`
- [\[P0901R8\]](#): Size Feedback In `operator new`

- [\[P2212R1\]](#): Relax Requirements For `time_point::clock`
- [\[P2166R1\]](#): Prohibit `basic_string` And `basic_string_view` Construction From `nullptr`

§ References

§ Informative References

[N4771]

Jonathan Wakely. *Working Draft, C++ Extensions for Networking*. 8 October 2018. URL: <https://wg21.link/n4771>

[P0009R14]

Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. *MDSPAN*. 15 November 2021. URL: <https://wg21.link/p0009r14>

[P0009R16]

Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. *MDSPAN*. 16 March 2022. URL: <https://wg21.link/p0009r16>

[P0323R10]

JF Bastien, Vicente Botet. *std::expected*. 15 April 2021. URL: <https://wg21.link/p0323r10>

[P0493R3]

Al Grant, Bronek Kozicki, Tim Northover. *Atomic maximum/minimum*. 18 December 2021. URL: <https://wg21.link/p0493r3>

[P0533R9]

Oliver Rosten, Edward Rosten. *constexpr for cmath and cstdlib*. 12 November 2021. URL: <https://wg21.link/p0533r9>

[P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

[P0627R6]

Jens Maurer. *Function to mark unreachable code*. 25 October 2021. URL: <https://wg21.link/p0627r6>

[P0901R8]

Chris Kennelly, Andrew Hunter. *Size feedback in operator new*. 15 December 2020. URL: <https://wg21.link/p0901r8>

[P1000R4]

Herb Sutter. *C++ IS schedule*. 14 February 2020. URL: <https://wg21.link/p1000r4>

[P1072R7]

Chris Kennelly, Mark Zeren. *basic_string::resize_and_overwrite*. 15 February 2021. URL: <https://wg21.link/p1072r7>

[P1206R6]

Corentin Jabot, Eric Niebler, Casey Carter. *[Conversions from ranges to containers](https://wg21.link/p1206r6)*. 3 August 2021. URL: <https://wg21.link/p1206r6>

[P1238R0]

Tom Honermann, Corentin Jabot, JeanHeyd Meneide, Mark Zeren, Martinho Fernandes, Peter Bindels, Steve Downey, Zach Laine. *[SG16: Unicode Direction](https://wg21.link/p1238r0)*. 8 October 2018. URL: <https://wg21.link/p1238r0>

[P1385R6]

Guy Davidson, Bob Steagall. *[A proposal to add linear algebra support to the C++ standard library](https://wg21.link/p1385r6)*. 1 March 2020. URL: <https://wg21.link/p1385r6>

[P1467R7]

David Olsen, Michał Dominiak, Ilya Burylov. *[Extended floating-point types and standard names](https://wg21.link/p1467r7)*. 24 November 2021. URL: <https://wg21.link/p1467r7>

[P1642R5]

Ben Craig. *[Freestanding Library: Easy \[utilities\], \[ranges\], and \[iterators\]](https://wg21.link/p1642r5)*. 10 December 2020. URL: <https://wg21.link/p1642r5>

[P1673R6]

Mark Hoemmen, Daisy Hollman, Christian Trott, Daniel Sunderland, Nevin Liber, Alicia Klinvex, Li-Ta Lo, Damien Lebrun-Grandie, Graham Lopez, Peter Caday, Sarah Knepper, Piotr Luszczek, Timothy Costa. *[A free function linear algebra interface based on the BLAS](https://wg21.link/p1673r6)*. 15 December 2021. URL: <https://wg21.link/p1673r6>

[P1684R1]

Christian Trott, D. S. Hollman, Mark Hoemmen, Daniel Sunderland. *[mdarray: An Owning Multidimensional Array Analog of mdspan](https://wg21.link/p1684r1)*. 21 March 2022. URL: <https://wg21.link/p1684r1>

[P1841R1]

Walter E Brown. *[Wording for Individually Specializable Numeric Traits](https://wg21.link/p1841r1)*. 15 May 2020. URL: <https://wg21.link/p1841r1>

[P1885R8]

Corentin Jabot, Peter Brett. *[Naming Text Encodings to Demystify Them](https://wg21.link/p1885r8)*. 13 October 2021. URL: <https://wg21.link/p1885r8>

[P1899R2]

Christopher Di Bella, Tim Song. *[stride_view](https://wg21.link/p1899r2)*. 23 December 2021. URL: <https://wg21.link/p1899r2>

[P1951R0]

Logan R. Smith. *[Default Arguments for pair's Forwarding Constructor](https://wg21.link/p1951r0)*. 17 November 2019. URL: <https://wg21.link/p1951r0>

[P2077R2]

Konstantin Boyarinov, Sergey Vinogradov, Ruslan Arutyunyan. *[Heterogeneous erasure overloads for associative containers](https://wg21.link/p2077r2)*. 15 December 2020. URL: <https://wg21.link/p2077r2>

[P2093R13]

Victor Zverovich. *[Formatted output](https://wg21.link/p2093r13)*. 9 February 2022. URL: <https://wg21.link/p2093r13>

[P2136R2]

Zhihao Yuan. *[invoke_r](https://wg21.link/p2136r2)*. 6 December 2020. URL: <https://wg21.link/p2136r2>

[P2138R3]

Ville Voutilainen. *Rules of Design <=> Specification engagement*. 15 September 2020. URL: <https://wg21.link/p2138r3>

[P2145R1]

Bryce Adelstein Lelbach, Titus Winters, Fabio Fracassi, Billy Baker, Nevin Liber, JF Bastien, David Stone, Botond Ballo, Tom Honermann. *Evolving C++ Remotely*. 15 September 2020. URL: <https://wg21.link/p2145r1>

[P2165R3]

Corentin Jabot. *Compatibility between tuple, pair and tuple-like objects*. 18 January 2022. URL: <https://wg21.link/p2165r3>

[P2166R1]

Yuriy Chernyshov. *A Proposal to Prohibit std::basic_string and std::basic_string_view construction from nullptr*. 7 September 2020. URL: <https://wg21.link/p2166r1>

[P2195R2]

Bryce Adelstein Lelbach. *Electronic Straw Polls*. 28 January 2021. URL: <https://wg21.link/p2195r2>

[P2198R3]

Ben Craig. *Freestanding Feature-Test Macros and Implementation-Defined Extensions*. 12 November 2021. URL: <https://wg21.link/p2198r3>

[P2210R2]

Barry Revzin. *Superior String Splitting*. 5 March 2021. URL: <https://wg21.link/p2210r2>

[P2212R1]

Alexey Dmitriev, Howard Hinnant. *Relax Requirements for time_point::clock*. 14 September 2020. URL: <https://wg21.link/p2212r1>

[P2214R2]

Barry Revzin, Conor Hoekstra, Tim Song. *A Plan for C++23 Ranges*. 18 February 2022. URL: <https://wg21.link/p2214r2>

[P2216R2]

Victor Zverovich. *std::format improvements*. 15 January 2021. URL: <https://wg21.link/p2216r2>

[P2231R0]

Barry Revzin. *Add further constexpr support for optional/variant*. 14 October 2020. URL: <https://wg21.link/p2231r0>

[P2247R1]

Bryce Adelstein Lelbach, Fabio Fracassi, Ben Craig, Billy Baker, Nevin Liber, Inbal Levi, Corentin Jabot. *2020 Library Evolution Report*. 3 December 2020. URL: <https://wg21.link/p2247r1>

[P2248R4]

Giuseppe D'Angelo. *Enabling list-initialization for algorithms*. 3 January 2022. URL: <https://wg21.link/p2248r4>

[P2249R1]

Giuseppe D'Angelo. *Mixed comparisons for smart pointers*. 14 July 2021. URL: <https://wg21.link/p2249r1>

[P2251R1]

Nevin Liber. *Require span & basic_string_view to be Trivially Copyable*. 19 March 2021. URL: <https://wg21.link/p2251r1>

[P2255R2]

Tim Song. *A type trait to detect reference binding to temporary*. 14 October 2021. URL: <https://wg21.link/p2255r2>

[P2265R0]

Kevlin Henney. *Renaming any_invocable*. 6 December 2020. URL: <https://wg21.link/p2265r0>

[P2273R2]

Andreas Fertig. *Making std::unique_ptr constexpr*. 5 July 2021. URL: <https://wg21.link/p2273r2>

[P2278R2]

Barry Revzin. *cbegin should always return a constant iterator*. 17 November 2021. URL: <https://wg21.link/p2278r2>

[P2286R6]

Barry Revzin. *Formatting Ranges*. 19 January 2022. URL: <https://wg21.link/p2286r6>

[P2291R2]

Daniil Goncharov, Karaev Alexander. *Add Constexpr Modifiers to Functions to chars and from chars for Integral Types in Header*. 17 August 2021. URL: <https://wg21.link/p2291r2>

[P2300R4]

Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach. *std::execution*. 19 January 2022. URL: <https://wg21.link/p2300r4>

[P2301R0]

Steve Downey. *Add a pmr alias for std::stacktrace*. 15 February 2021. URL: <https://wg21.link/p2301r0>

[P2302R2]

Christopher Di Bella. *std::ranges::contains*. 12 December 2021. URL: <https://wg21.link/p2302r2>

[P2321R1]

Tim Song. *zip*. 11 April 2021. URL: <https://wg21.link/p2321r1>

[P2322R5]

Barry Revzin. *ranges::fold*. 18 October 2021. URL: <https://wg21.link/p2322r5>

[P2325R2]

Barry Revzin. *Views should not be required to be default constructible*. 23 April 2021. URL: <https://wg21.link/p2325r2>

[P2328R0]

Tim Song. *join_view should join all views of ranges*. 15 March 2021. URL: <https://wg21.link/p2328r0>

[P2340R0]

Thomas Köppe. *Clarifying the status of the ‘C headers’*. 15 March 2021. URL: <https://wg21.link/p2340r0>

[P2351R0]

Hana Dusíková. *Mark all library static cast wrappers as [[nodiscard]]*. 25 April 2021. URL: <https://wg21.link/p2351r0>

[P2363R3]

Konstantin Boyarinov, Sergey Vinogradov, Ruslan Arutyunyan. *[Extending associative containers with the remaining heterogeneous overloads](https://wg21.link/p2363r3)*. 19 January 2022. URL: <https://wg21.link/p2363r3>

[P2372R1]

Victor Zverovich, Corentin Jabot. *[Fixing locale handling in chrono formatters](https://wg21.link/p2372r1)*. 13 May 2021. URL: <https://wg21.link/p2372r1>

[P2374R3]

Sy Brand, Michał Dominiak. *[views::cartesian_product](https://wg21.link/p2374r3)*. 13 December 2021. URL: <https://wg21.link/p2374r3>

[P2387R2]

Barry Revzin. *[Pipe support for user-defined range adaptors](https://wg21.link/p2387r2)*. 18 October 2021. URL: <https://wg21.link/p2387r2>

[P2396R0]

David Goldblatt. *[Concurrency TS 2 fixes](https://wg21.link/p2396r0)*. 14 June 2021. URL: <https://wg21.link/p2396r0>

[P2400R0]

Bryce Adelstein Lelbach. *[Library Evolution Report](https://wg21.link/p2400r0)*. 27 February 2021. URL: <https://wg21.link/p2400r0>

[P2400R1]

Bryce Adelstein Lelbach. *[Library Evolution Report: 2021-02-23 to 2021-05-25](https://wg21.link/p2400r1)*. 8 June 2021. URL: <https://wg21.link/p2400r1>

[P2400R2]

Bryce Adelstein Lelbach. *[Library Evolution Report: 2021-06-01 to 2021-09-20](https://wg21.link/p2400r2)*. 28 September 2021. URL: <https://wg21.link/p2400r2>

[P2404R2]

Justin Bassett. *[Move-only types for equality_comparable_with, totally_ordered_with, and three_way_comparable_with](https://wg21.link/p2404r2)*. 19 January 2022. URL: <https://wg21.link/p2404r2>

[P2405R0]

Justin Bassett. *[nullopt_t and nullptr_t should both have operator and operator==](https://wg21.link/p2405r0)*. 15 July 2021. URL: <https://wg21.link/p2405r0>

[P2407R0]

Emil Meissner, Ben Craig. *[Freestanding Library: Partial Classes](https://wg21.link/p2407r0)*. 12 July 2021. URL: <https://wg21.link/p2407r0>

[P2408R4]

David Olsen. *[Ranges iterators as inputs to non-Ranges algorithms](https://wg21.link/p2408r4)*. 16 November 2021. URL: <https://wg21.link/p2408r4>

[P2415R1]

Barry Revzin, Tim Song. *[What is a view?](https://wg21.link/p2415r1)*. 16 August 2021. URL: <https://wg21.link/p2415r1>

[P2417R0]

Daniil Goncharov. *[A more constexpr bitset](https://wg21.link/p2417r0)*. 24 July 2021. URL: <https://wg21.link/p2417r0>

[P2418R0]

Victor Zverovich. *[Add support for std::generator-like types to std::format](https://wg21.link/p2418r0)*. 8 August 2021. URL: <https://wg21.link/p2418r0>

[P2419R1]

Victor Zverovich, Peter Brett. *Clarify handling of encodings in localized formatting of chrono types*. 19 September 2021. URL: <https://wg21.link/p2419r1>

[P2432R0]

Nicolai Josuttis. *Fixing istream view*. 27 August 2021. URL: <https://wg21.link/p2432r0>

[P2438R1]

Federico Kircheis, Tomasz Kamiński. *std::string::substr() &&*. 20211130. URL: <https://wg21.link/p2438r1>

[P2440R0]

Tim Song. *ranges::iota, ranges::shift_left, and ranges::shift_right*. 13 September 2021. URL: <https://wg21.link/p2440r0>

[P2441R1]

Barry Revzin. *views::join_with*. 17 November 2021. URL: <https://wg21.link/p2441r1>

[P2442R0]

Tim Song. *Windowing range adaptors: views::chunk and views::slide*. 14 September 2021. URL: <https://wg21.link/p2442r0>

[P2443R0]

Tim Song. *views::chunk_by*. 15 September 2021. URL: <https://wg21.link/p2443r0>

[P2444R0]

Christopher Kohlhoff. *The Asio asynchronous model*. 15 September 2021. URL: <https://wg21.link/p2444r0>

[P2445R0]

Gašper Ažman. *forward_like*. 12 October 2021. URL: <https://wg21.link/p2445r0>

[P2446R1]

Barry Revzin. *views::all_move*. 17 November 2021. URL: <https://wg21.link/p2446r1>

[P2447R0]

Federico Kircheis. *std::span and the missing constructor*. 14 October 2021. URL: <https://wg21.link/p2447r0>

[P2453R0]

Bryce Adelstein Lelbach. *2021 October Library Evolution Poll Outcomes*. 15 February 2022. URL: <https://wg21.link/p2453r0>

[P2455R0]

Bryce Adelstein Lelbach. *2021 November Library Evolution Poll Outcomes*. 8 December 2021. URL: <https://wg21.link/p2455r0>

[P2457R0]

Bryce Adelstein Lelbach. *2021 December Library Evolution Poll Outcomes*. 14 January 2022. URL: <https://wg21.link/p2457r0>

[P2459R0]

Bryce Adelstein Lelbach, Fabio Fracassi, Ben Craig. *2022 January Library Evolution Poll Outcomes*. 15 February 2022. URL: <https://wg21.link/p2459r0>

[P2460R0]

Corentin Jabot. *Relax requirements on wchar_t to match existing practices*. 9 October 2021. URL: <https://wg21.link/p2460r0>

[P2464R0]

Ville Voutilainen. *[Ruminations on networking and executors](https://wg21.link/p2464r0)*. 29 September 2021. URL: <https://wg21.link/p2464r0>

[P2465R1]

Stephan T. Lavavej, Gabriel Dos Reis, Bjarne Stroustrup, Jonathan Wakely. *[Standard Library Modules std and std.compat](https://wg21.link/p2465r1)*. 13 October 2021. URL: <https://wg21.link/p2465r1>

[P2467R0]

Jonathan Wakely. *[Support exclusive mode for fstreams](https://wg21.link/p2467r0)*. 15 November 2021. URL: <https://wg21.link/p2467r0>

[P2469R0]

Christopher Kohlhoff, Jamie Allsop, Vinnie Falco, Richard Hodges, Klemens Morgenstern. *[Response to P2464: The Networking TS is baked, P2300 Sender/Receiver is not.](https://wg21.link/p2469r0)* 4 October 2021. URL: <https://wg21.link/p2469r0>

[P2474R1]

Michał Dominiak. *[views::repeat](https://wg21.link/p2474r1)*. 18 January 2022. URL: <https://wg21.link/p2474r1>

[P2485R0]

Jonathan Wakely. *[Do not add value_exists and value_or to C++23](https://wg21.link/p2485r0)*. 1 November 2021. URL: <https://wg21.link/p2485r0>

[P2486R1]

Kirk Shoop. *[Structured naming for function object and CPO values](https://wg21.link/p2486r1)*. 16 December 2021. URL: <https://wg21.link/p2486r1>

[P2489R0]

Bryce Adelstein Lelbach. *[Library Evolution Plan for Completing C++23](https://wg21.link/p2489r0)*. 9 November 2021. URL: <https://wg21.link/p2489r0>

[P2494R1]

Michał Dominiak. *[Relaxing range adaptors to allow for move only types](https://wg21.link/p2494r1)*. 17 January 2022. URL: <https://wg21.link/p2494r1>

[P2502R1]

Casey Carter. *[std::generator: Synchronous Coroutine Generator for Ranges](https://wg21.link/p2502r1)*. 25 January 2022. URL: <https://wg21.link/p2502r1>

[P2508R1]

Barry Revzin. *[Exposing std::basic-format-string](https://wg21.link/p2508r1)*. 18 January 2022. URL: <https://wg21.link/p2508r1>

[P2529R0]

Mathias Stearn. *[generator should have T&& reference type](https://wg21.link/p2529r0)*. 25 January 2022. URL: <https://wg21.link/p2529r0>

[P2553R0]

Christian Trott, Damien Lebrun-Grandie, Mark Hoemmen, Dan Sunderland. *[Make mdspan size_type controllable](https://wg21.link/p2553r0)*. 14 February 2022. URL: <https://wg21.link/p2553r0>

[P2554R0]

Christian Trott, Damien Lebrun-Grandie, Mark Hoemmen, K. R. Walker, Dan Sunderland. *[C-Array Interoperability of MDSpan](https://wg21.link/p2554r0)*. 15 February 2022. URL: <https://wg21.link/p2554r0>

[P2559R0]

Bryce Adelstein LeBach. *[Plan for Concurrency Technical Specification Version 2](https://wg21.link/P2559R0)*. 2022-02-28. URL:
<https://wg21.link/P2559R0>